

Gourmet Bites — Competition Technical Guide

A Deep Dive into Every Concept Used in All Languages

SECTION 1: PYTHON (Backend — App.py)

Python is used to power the **server-side logic** of this project. It stores the menu data and does all the mathematical calculations safely.

1.1 — Importing a Framework

`python from flask import Flask, render_template, jsonify, request` **import** brings in external code (libraries) so we can reuse it. - `Flask` — the main class to create our web server - `render_template` — sends an HTML file to the browser - `jsonify` — converts a Python dictionary into a JSON response - `request` — lets us read data sent by the browser (like cart items)

1.2 — Creating the Flask App Object

`python app = Flask(__name__)` `app.json.sort_keys = False` - `Flask(__name__)` creates one central "app" object. `__name__` tells Flask where our file is located. - `sort_keys = False` tells Flask NOT to alphabetically sort the JSON keys, so our category order is preserved exactly.

1.3 — Dictionaries and Lists (Data Structures)

`python MENU = { "Starters": [ {"id": "s1", "name": "Spring Rolls", "price": 5.99, "description": "Crispy vegetable spring rolls"}, ... ], "Beverages": [ ... ] }` - A **Dictionary** `{}` stores data as **key: value** pairs. Here, the key is the category name ("Starters") and the value is a list of items. - A **List** `[]` is an ordered collection. Each item inside is another dictionary with 4 keys: `id`, `name`, `price`, `description`. - This is a **nested data structure** — a dictionary containing lists of dictionaries.

1.4 — Dictionary Comprehension (Advanced Python)

`python FLAT_MENU = {item["id"]: item for category in MENU.values() for item in category}` This is one of the most powerful lines in the project. It's called a **dictionary comprehension**. - `MENU.values()` — gets all the category lists (Starters, Beverages, etc.) - `for category in MENU.values()` — loops through each category - `for item in category` — loops through each item inside the category - `item["id"]: item` — creates a new flat dictionary where the key is the item ID and the value is the full item

Why? Instead of searching through all 5 categories every time, we can look up any item directly by its ID in $O(1)$ time.

1.5 — Route Decorators (URL Endpoints)

`python @app.route("/") def index(): return render_template("index.html")` - `@app.route("/")` is a **decorator**. It tells Flask "when someone visits the homepage `/`, run the function below." - `render_template("index.html")` finds the `index.html` file in the `templates` folder and sends it to the browser.

`python @app.route("/api/menu") def get_menu(): return jsonify(MENU)` - This creates an **API endpoint** at `/api/menu`. - `jsonify(MENU)` converts the Python `MENU` dictionary into JSON format and sends it.

1.6 — Handling POST Requests & Input Validation

`python @app.route("/api/bill", methods=["POST"]) def calculate_bill(): data = request.json cart = data.get("cart", [])` - `methods=["POST"]` means this route only accepts POST requests (data being sent TO the server). - `request.json` reads the JSON body that was sent from the browser. - `.get("cart", [])` — safely retrieves the `"cart"` key. If it doesn't exist, it returns an empty list `[]` instead of crashing.

1.7 — Loops and Arithmetic (Bill Calculation)

`python subtotal = 0 for cart_item in cart: item_id = cart_item.get("id") quantity = cart_item.get("quantity", 1) if item_id in FLAT_MENU: menu_item = FLAT_MENU[item_id] item_total = menu_item["price"] * quantity subtotal += item_total` - `for cart_item in cart` — loops through every item the customer ordered. - `if item_id in FLAT_MENU` — **security check!** We verify the item exists on our server before trusting its price. A customer cannot change the price in the browser and cheat the system. - `subtotal += item_total` — accumulates the running total using the `+=` **compound operator**.

`python tax_rate = 0.08 tax = subtotal * tax_rate total = subtotal + tax` - A simple 8% tax calculation applied on the server side.

1.8 — Returning JSON Response

`python return jsonify({ "items": items, "subtotal": round(subtotal, 2), "tax": round(tax, 2), "total": round(total, 2) })` - `round(value, 2)` ensures prices only have 2 decimal places (like \$12.99, not \$12.9999999). - `jsonify` packages the Python dictionary into a proper JSON HTTP response that the browser can read.

1.9 — Entry Point

`python if __name__ == "__main__": app.run(debug=True, port=5000)` - `if __name__ == "__main__"` is a standard Python pattern meaning: "only run this code if this file is executed directly, not imported." - `debug=True` — Flask automatically reloads when we save changes to the code. - `port=5000` — the server listens on port 5000, making it available at `http://127.0.0.1:5000`.

SECTION 2: HTML (`index.html`)

HTML provides the **structural skeleton** of the webpage. Every visual element starts as an HTML element.

2.1 — Semantic HTML5 Structure

```
```html
```

```
... ..
```

```
`` -<!DOCTYPE html>- tells the browser this is an HTML5 document. -- supports all special characters and emojis. -- makes the page responsive on mobile screens. -
```

```
,,` are semantic tags — they describe the purpose of the content inside them.
```

---

### 2.2 — External Resource Linking

```
html <link href="https://fonts.googleapis.com/css2?family=Outfit..." rel="stylesheet"> <link rel="stylesheet"
```

```
href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css"> <link
```

```
rel="stylesheet" href="/static/p.css?v=2"> - The first link loads the Outfit font from Google Fonts. - The second link loads Font Awesome icon library from a CDN (Content Delivery Network). - The third link loads our own custom CSS file. ?v=2 is a cache-busting trick to force the browser to download the latest version.
```

---

### 2.3 — Dynamic Placeholder Containers

```
```html
```

```
Loading menu...
```

```
`` - This is an empty container. At page load it shows "Loading menu..." - JavaScript will later inject all the food item cards into this container dynamically, without refreshing the page. -id="menuGrid" is used by JavaScript to target this exact element.
```

2.4 — The Welcome Overlay

```
```html
```

```
Let's Order!
```

```
`` - This
```

```
covers the entire screen (done through CSSposition: fixed). - It blocks the user from interacting with the menu until they enter a valid name. - JavaScript listens for the button click and then hides this overlay.
```

---

## 2.5 — The Confirm Modal

```
```html
```

Confirm Order

Are you sure you want to place this order?

Cancel Yes, Place Order

```
`` - This is a modal dialog — a popup that appears on top of the page. - It is hidden by default and becomes visible when JavaScript adds the show` CSS class to it. - It creates a two-step confirmation before finalizing an order.
```

```
---
```

SECTION 3: CSS (`p.css`)

CSS is responsible for the **entire visual design** — colors, layout, animations, and responsiveness.

3.1 — CSS Variables (Design System)

```
css :root { --bg-dark: #eee4d1; --bg-card: #fffdf5; --primary: #6f4e37; --primary-hover: #4a3322; --accent: #eab308; --glass-border: rgba(111, 78, 55, 0.45); } - :root is a pseudo-class that targets the very top of the document. - Variables declared here (e.g., --primary) can be used anywhere in the CSS with var(--primary). - Why use variables? If you want to change the entire theme, you only change the color in ONE place. This is called a Design System or Design Tokens.
```

3.2 — Glassmorphism Effect

```
css .navbar { background: rgba(255, 253, 245, 0.85); backdrop-filter: blur(12px); -webkit-backdrop-filter: blur(12px); border-bottom: 1px solid var(--glass-border); } -
```

Glassmorphism is a modern design trend that makes elements look like frosted glass. - `rgba(255, 253, 245, 0.85)` — a color with **alpha (transparency)**. The `0.85` means 85% opaque. - `backdrop-filter: blur(12px)` — blurs everything **behind** this element. - `-webkit-backdrop-filter` — a vendor prefix for Safari browser compatibility.

3.3 — CSS Grid (Responsive Menu Layout)

```
css .menu-grid { display: grid; grid-template-columns: repeat(auto-fill, minmax(280px, 1fr)); gap: 25px; } - display: grid activates CSS Grid layout. - repeat(auto-fill, minmax(280px, 1fr)) — this is the most powerful CSS line in the project: - It creates as many columns as can fit in the available space. - Each column is minimum 280px wide, maximum 1fr (equal fraction of remaining space). - This alone makes the layout fully responsive — on a wide screen you get 4 columns, on mobile you get 1.
```

3.4 — Flexbox (Navigation & Alignment)

`css .navbar { display: flex; justify-content: space-between; align-items: center; }` - `display: flex` activates Flexbox — a one-dimensional layout system (row or column). - `justify-content: space-between` — pushes the logo to the left and the cart icon to the right. - `align-items: center` — vertically centers both items in the middle.

3.5 — CSS Animations with `@keyframes`

```
css @keyframes logoFade { 0%, 100% { opacity: 1; } 50% { opacity: 0.6; } }
```

`.logo { animation: logoFade 3s ease-in-out infinite; }` - `@keyframes` defines a custom animation with named steps (0%, 50%, 100%). - `animation: logoFade 3s ease-in-out infinite` applies it: - `logoFade` — name of our keyframe - `3s` — takes 3 seconds to complete - `ease-in-out` — starts slow, speeds up, slows down (smooth) - `infinite` — loops forever

```
css @keyframes heroBgShift { 0%, 25% { background: url('image1.jpg'); } 33%, 58% { background: url('image2.jpg'); } 66%, 91% { background: url('image3.jpg'); } 100% { background: url('image1.jpg'); } }
```

 - This shifts through 3 different background food images over 25 seconds, creating a slideshow effect.

3.6 — CSS Transitions (Hover Effects)

```
css .menu-item { transition: var(--transition); } .menu-item:hover { transform: translateY(-5px); box-shadow: 0 15px 30px rgba(0,0,0,0.08); }
```

 - `transition` makes changes animate smoothly instead of jumping instantly. - `:hover` is a **pseudo-class** — styles applied when the mouse hovers over the element. - `transform: translateY(-5px)` moves the card 5 pixels upward, creating a "lift" effect.

3.7 — CSS Background Watermark (Receipt Logo)

```
css .bill-content { background: linear-gradient(rgba(255, 253, 245, 0.75), rgba(255, 253, 245, 0.75)), url('/static/receipt_bg.png') center/50% no-repeat; }
```

 - Two backgrounds are layered. CSS processes them **top to bottom** — first the gradient overlay, then the logo image. - The semi-transparent gradient sits on top of the logo, making it appear as a light watermark. - `center/50% no-repeat` — centers the logo and sets it to 50% of the container size.

SECTION 4: JAVASCRIPT (`app.js`)

JavaScript is the **brain of the browser** — it handles all user interactions, communicates with the server, and updates the page without refreshing.

4.1 — DOM Content Loaded Event

```
javascript document.addEventListener('DOMContentLoaded', () => { // All our code goes here });
```

`DOMContentLoaded` fires only after the full HTML is loaded and parsed. - This ensures we don't try to access buttons or divs before they exist on the page. - `() => {}` is an **Arrow Function** — a modern, concise way to write functions in JavaScript.

4.2 — DOM Querying (Getting HTML Elements)

```
javascript const menuGrid = document.getElementById('menuGrid'); const categoryFilter = document.getElementById('categoryFilter'); const checkoutBtn = document.getElementById('checkoutBtn');
```

`document.getElementById('id')` finds the exact HTML element with that ID. - `const` declares a variable that cannot be reassigned (a constant reference). - We store references to HTML elements so we can interact with them throughout our code.

4.3 — State Management with Variables

```
javascript let userName = ''; let menuData = {}; let cart = []; // Array of {id, name, price, quantity}
```

`let` is used for variables that **will change** over time. - `cart` is an array that holds all items the customer has added. Every click on "Add to Cart" updates this array. This is the **application state**.

4.4 — Regular Expressions (Input Validation)

```
javascript const isValidName = /^[a-zA-Z\s]+$/.test(name);
```

`^[a-zA-Z\s]+$` is a **Regular Expression (Regex)** — a pattern used to match text. - `^` — start of the string - `[a-zA-Z\s]` — only lowercase letters, uppercase letters, or spaces - `+` — one or more characters - `$` — end of the string - `.test(name)` checks if the name matches this pattern and returns `true` or `false`. - **This prevents users from entering numbers or symbols as their name.**

4.5 — The Fetch API (Talking to the Backend)

```
javascript fetch('/api/menu').then(res => res.json()).then(data => { menuData = data; initApp(); }).catch(err => { console.error('Error fetching menu:', err); });
```

`fetch()` makes an HTTP GET request to the Flask backend at `/api/menu`. - It is **asynchronous** — the browser doesn't freeze waiting for the data. - `.then()` is called when the response arrives (a **Promise chain**). - `res.json()` converts the raw response into a JavaScript object. - `.catch()` handles any network errors gracefully.

4.6 — Dynamic DOM Creation (renderMenu)

```
javascript itemsToRender.forEach(item => { const el = document.createElement('div'); el.className = 'menu-item'; el.innerHTML = `<span class="item-name">${item.name}</span> <span class="item-price">${item.price.toFixed(2)}</span> `; menuGrid.appendChild(el); });
```

`document.createElement('div')` creates a brand new HTML element in memory. - `el.innerHTML = `...`` uses a

Template Literal (backtick string) to inject dynamic HTML. `${item.name}` embeds the variable value directly. `-menuGrid.appendChild(el)` attaches the new element to the page — this is how 50 food items appear on screen from a blank container.

4.7 — Array Methods (Cart Logic)

```
``javascript // Find existing item const existing = cart.find(item => item.id === id);

// Add new item cart.push({ id, name, price, quantity: 1 });

// Remove item cart = cart.filter(i => i.id !== id); `` - .find() searches the array and returns the first element matching the condition. -.push() adds a new element to the end of the array. -.filter() creates a new array with only elements that pass the test. Used to remove items. -{ id, name, price } ES6 Shorthand Property — equivalent to { id: id, name: name, price: price }.
```

4.8 — classList API (Showing/Hiding Elements)

```
``javascript // Show the cart sidebar cartSidebar.classList.add('open');

// Hide the confirm modal confirmModal.classList.remove('show'); `` - classList is a property of every HTML element that manages its CSS classes. -.add('open') adds the open class, which our CSS uses to slide the sidebar into view. -.remove('show') removes the show class, making the modal invisible again. - This is a core pattern in modern web development — toggle CSS classes with JavaScript.
```

4.9 — Sending Data with POST Fetch (Checkout)

```
javascript fetch('/api/bill', { method: 'POST', headers: { 'Content-Type': 'application/json' },
body: JSON.stringify({ cart: cart }) }) .then(res => res.json()) .then(data => { showBill(data);
}); - method: 'POST' — we are sending data to the server. - headers: { 'Content-Type': 'application/json' } — tells the server we are sending JSON. - JSON.stringify({ cart: cart }) — converts our JavaScript cart array into a JSON string to transmit it. - The server receives this, calculates the bill, and sends back the receipt data.
```

4.10 — setTimeout (Timed Actions)

```
javascript setTimeout(() => { customerNameInput.style.borderColor = '';
customerNameInput.placeholder = 'Enter your name...'; }, 2000); - setTimeout(function, delay) runs a function once after a delay in milliseconds. - 2000ms = 2 seconds. This resets the red error border after showing it for 2 seconds. - This creates a smooth user experience without jarring instant resets.
```

SUMMARY: The Technology Stack

Layer	Technology	Purpose
Backend	Python + Flask	Server, API endpoints, bill calculation
Data Format	JSON	Communication between frontend & backend
Structure	HTML5	Page skeleton and semantic elements
Styling	CSS3	Design, layout, animations, responsiveness
Interactivity	JavaScript (ES6+)	Dynamic rendering, cart logic, API calls
Icons	Font Awesome CDN	All icons in the UI
Fonts	Google Fonts (Outfit)	Premium typography